

Incident Postmortem Copilot

Every step, start to finish, including the current (2026) way to build the tool in Step 6 — an agent flow filing a parent work item and child action items in Azure DevOps.

i

What You're Building

An agent that drafts blameless postmortems and files them as Azure DevOps work items, grounded in one data file, acting through one governed tool.

AGENT NAME	Incident Postmortem Copilot
ORCHESTRATION	Generative (AI-led)
KNOWLEDGE	Tech_Incidents_Data.xlsx
CUSTOM TOPIC	Draft Incident Postmortem
TOOL	File Incident Postmortem in Azure DevOps (agent flow → ADO + Teams)

Files & Where They Go

FILE / ITEM	GOES TO	USED IN
Tech_Incidents_Data.xlsx	Copilot Studio → Knowledge tab → Add knowledge → File	Step 3
Azure DevOps connection	Authorize the connector, confirm the 'Incident Postmortem' work item type exists in your project	Step 6
PostmortemFiled_AdaptiveCard.json	Paste into the flow's Adaptive Card field	Step 6

Before you start — confirm Environment Maker permission, an authorized Azure DevOps connection, and that your ADO project has (or can add) an 'Incident Postmortem' work item type.

Set Up the Agent

Steps 1–3 — create the agent shell, pick how it reasons, and ground it in data.

1 Create Your Copilot Studio Agent

Open Copilot Studio (copilotstudio.microsoft.com), select **Create** → **New agent**, then configure:

AGENT NAME	Incident Postmortem Copilot
DESCRIPTION	Drafts engineering incident postmortems from incident logs, identifies recurring root-cause patterns, suggests action items, and never assigns individual blame.
SOLUTION / ENVIRONMENT	Engineering Operations · Default environment
ICON	Use a recognisable colour and emoji or upload a brand icon

2 Choose Orchestration Mode

Under **Settings** → **Generative AI**, use **Generative Orchestration** (AI-led) so the agent can blend knowledge lookups with tool calls for incident response and blameless postmortems.

3 Add Knowledge

Knowledge tab → **Add knowledge** → **File** → upload **Tech_Incidents_Data.xlsx**. Contains:

- Incident metadata (incident ID, severity SEV1-4, service, environment Prod/Stage)
- Timeline (start UTC, detected UTC, resolved UTC, MTTR minutes)
- Root cause (root cause category, summary)
- Follow-up (action items open, owning team)

You can layer additional sources later — SharePoint sites, Dataverse, public web, or Graph connectors.

Behavior: Instructions & Topic

Steps 4–5 — tell the agent how to behave, then give it a reusable conversation flow.

4 Add Agent Instructions

Overview → Instructions — paste exactly:

You are the Incident Postmortem Copilot. You help two audiences:

- 1) Engineers writing a postmortem after an incident.
- 2) Engineering leaders reviewing root-cause patterns across the estate.

Rules:

- Use the uploaded incidents dataset as your single source of truth.
- Blameless language always – describe systems, signals and decisions, never individuals. Replace any name in source notes with 'on-call engineer' or 'owning team' before quoting back.
- Quote incident IDs and timestamps verbatim – no rounding.
- For postmortem filing, call the tool – never just claim it was filed.
- Highlight recurring root cause categories (3+ in 60 days).

5 Add a Custom Topic

Topics → Add a topic → Create from description:

Topic name: Draft Incident Postmortem

Description: Walks an engineer through drafting a blameless postmortem for a chosen incident, then files it as a work item in Azure DevOps for the owning team.

Trigger phrases:

- Draft a postmortem for INC-2451
- Help me write up the SEV2 from yesterday
- Postmortem for the checkout outage
- Start an incident retro

Conversation steps:

- Ask for the IncidentID – confirm one row matches in knowledge
- Render structured draft: Summary · Timeline · Impact · Root cause · Action items
- Pull from row: StartUTC, DetectedUTC, ResolvedUTC, MTTRMinutes, RootCauseCategory
- Suggest 3-5 action items in blameless language, owned by team not person
- Read it back; ask the engineer to confirm or edit any section
- Call 'File Incident Postmortem in Azure DevOps' with the final draft

Build the Tool (Current Method)

Step 6 — updated for 2026, using current Copilot Studio tool-building.

Why this needs a flow, not a direct connector — the tool must create *one parent work item plus a child work item per action item*, which means looping over a list. That looping logic is exactly what an agent flow is for, even though the underlying action (Azure DevOps) is a single connector.

6a Build the Agent Flow

1. **Flows** → **New flow** → **Agent flow** (or describe the whole thing, including the loop, to the natural-language builder).
2. Trigger inputs: **IncidentID**, **Severity**, **OwningTeam**, **PostmortemBody**, **ActionItems** (array).
3. Action: **Azure DevOps — Create a work item** → type **Incident Postmortem** → title from IncidentID, description from PostmortemBody, team from OwningTeam.
4. Add an **Apply to each** loop over **ActionItems** → inside it, **Azure DevOps — Create a work item** (type Task) → linked as a child of the parent work item created above.
5. Action: **Teams — Post your own adaptive card as the Flow bot to a channel** → the owning team's channel → paste the contents of **PostmortemFiled_AdaptiveCard.json** into the Adaptive Card field, then replace each `{{Placeholder}}` with the matching field from Dynamic content.
6. **Respond to the agent** → output **WorkItemURL**. Confirm **Asynchronous response = Off**.
7. **Publish** the flow.

6b Attach It as a Tool

1. **Tools** → **Add a tool** → filter by **Flow** → select it → **Add and configure**.
2. **Name**: File Incident Postmortem in Azure DevOps. **Description**: "Creates a postmortem work item with action items as child tasks, linked to the incident."
3. Under **Completion**, respond using **WorkItemURL**.
4. **Save**.

Original spec, for reference — Tool: File Incident Postmortem in Azure DevOps. Type: Azure DevOps connector (Create work item — type 'Incident Postmortem'). Creates a postmortem work item on the owning team's board with the drafted summary, action items as child tasks, and links back to the incident ID. Inputs: Incident ID (string); Severity (SEV1 / SEV2 / SEV3 / SEV4); Owning team (string, from knowledge); Postmortem markdown body (string — sections Summary / Timeline / Impact / Root Cause); Action items list (string array — each becomes a child work item). Outcome: Returns the Azure DevOps work item URL and posts a card to the owning team's Teams channel for visibility.

Configure, Evaluate, Test, Publish

Steps 7–10 — lock down settings, prove quality, and ship it.

7 Configure Settings

Settings cog (top right) — four areas before real users touch the agent:

- **General** — confirm name, description, primary language
- **Generative AI** — orchestration mode, content moderation (start at Medium)
- **Authentication** — "Authenticate with Microsoft" so the agent runs as the signed-in user
- **Security** — review DLP policy, enabled channels, end-user authentication

8 Evaluate Your Agent

Analytics → **Evaluations** → **New evaluation** — "Incident Postmortem Copilot — Baseline v1". Score every response on Groundedness, Relevance, Accuracy.

Input: Draft a postmortem for INC-2451

Expect: Returns blameless structured draft, no names; timeline matches row verbatim.

Input: What root cause categories repeat in the last 60 days?

Expect: Counts RootCauseCategory occurrences within 60 days, flags any with count >= 3.

Input: File the postmortem for INC-2451 to Azure DevOps for the Checkout team

Expect: Triggers Draft Incident Postmortem topic and calls the Azure DevOps tool.

9 Test in the Test Pane

For each query, open **Activity map** to see: which topic/tool/knowledge fired, the inputs collected and outputs returned, citations to source rows, and per-step latency.

10 Publish to a Channel

Recommended for this build: Microsoft Teams (Engineering channels) — link postmortems back to team boards. Pin to each engineering team channel; surface in **Microsoft 365 Copilot** for leaders reviewing root cause patterns; embed on the Engineering Excellence portal; or expose the agent's tools via an **MCP server** to your existing dev tooling stack. Teams and M365 Copilot channels require tenant admin approval.

Try These Live

Sample queries to run once the agent is built — and what it replaces.

Postmortem Drafting

- Draft a postmortem for INC-2451.
- Give me a blameless summary of yesterday's SEV2.

Pattern Detection

- What root cause categories repeat in the last 60 days?
- Which service has the worst MTTR in production this quarter?

Governed Action

- File the postmortem for INC-2451 as a work item on the Checkout team's board.
- Try a paraphrase: 'ship the retro for the payment timeout incident'.

This agent replaces

- Engineers writing postmortems from a blank page
- Manual root-cause tagging in spreadsheets
- Action items lost in chat threads

With

- Consistent, blameless postmortems on every incident
- Action items tracked as work items, not memory
- Recurring root causes surfaced for leadership

Tip for real-world use — add an MCP server alongside the Azure DevOps tool so the agent can pull live metrics from your observability stack into the Impact section automatically. Run the evaluation set after every model upgrade.